

CSE 2218- Section: D
Data Structures and Algorithms-II Lab
Mid Practice Sheet

Summer 2026

Disclaimer

This practice sheet is prepared for exam preparation purposes.
Students should understand the algorithm and implementation logic
rather than memorize solutions.

Department of Computer Science and Engineering

Divide & Conquer

- Maximum Subarray Sum
- Merge Sort
- Quick Sort
- Inversion Count
- Kth Element
- Median
- Max/Min variations

Greedy

- Activity Selection
- Fractional Knapsack
- Scheduling variations

Dynamic Programming

- Fibonacci
- Coin Change
- 0/1 Knapsack
- Knapsack variations
- Basic recurrence problems

Part 1: Divide and Conquer Problems (15 Problems)

Instructions:

Write programs to solve the following problems using a divide-and-conquer strategy. Provide all required functions.

1. Divide and Conquer: Maximum Subarray Sum

Write a program to find the maximum sum of a contiguous subarray within a given array using a divide-and-conquer strategy. The algorithm should divide the array into smaller parts and calculate the maximum sum from the left part, right part, and crossing part. Provide all required functions.

Sample Input	Sample Output
Array: [-2, 3, 5, -1, 4, -6]	Maximum Sum: 11
Array: [1, -3, 4, -2, 5]	Maximum Sum: 7

2. Divide and Conquer: Maximum Subarray Product

Write a program to find the maximum product of a contiguous subarray using a divide-and-conquer strategy. The array may contain zero and positive integers. Provide all required functions.

Sample Input	Sample Output
Array: [2, 3, 0, 4, 5]	Maximum Product: 20
Array: [0, 1, 2, 3, 0, 4]	Maximum Product: 6

3. Divide and Conquer: Merge Sort

Write a program to sort an array in ascending order using the merge sort algorithm. The program should divide the array into smaller parts, sort them recursively, and merge the results. Provide all required functions.

Sample Input	Sample Output
Array: [8, 3, 5, 1, 9, 6]	Sorted Array: [1,3,5,6,8,9]
Array: [10,4,7,2,8]	Sorted Array: [2,4,7,8,10]

4. Divide and Conquer: Count Inversions

Write a program to count the number of inversions in an array using a merge-sort based divide-and-conquer approach. An inversion occurs when an element before another element is greater than it.

Provide all required functions.

Sample Input	Sample Output
Array: [8,4,2,1]	Number of Inversions: 6
Array: [1,3,2,5,4]	Number of Inversions: 2

5. Divide and Conquer: Quick Sort

Write a program to sort an array using the quick sort algorithm. Select a pivot element, partition the array, and recursively sort the subarrays.

Provide all required functions.

Sample Input	Sample Output
Array: [10,7,8,9,1,5]	Sorted Array: [1,5,7,8,9,10]
Array: [4,2,7,3,1]	Sorted Array: [1,2,3,4,7]

6. Divide and Conquer: Kth Smallest Element

Write a program to find the kth smallest element from an unsorted array using a partition-based divide-and-conquer approach.

Provide all required functions.

Sample Input	Sample Output
Array: [7,10,4,3,20,15], K=3	Kth Smallest Element: 7
Array: [12,3,5,7,4,19,26], K=2	Kth Smallest Element: 4

7. Divide and Conquer: Median of an Unsorted Array

Write a program to find the median value of an unsorted array using a divide-and-conquer partition technique.

Provide all required functions.

Sample Input	Sample Output
Array: [7,1,5,3,9]	Median: 5
Array: [10,2,8,4,6]	Median: 6

8. Divide and Conquer: Find Maximum Element

Write a program to find the maximum element from an array using a divide-and-conquer strategy.

Provide all required functions.

Sample Input	Sample Output
Array: [12,45,7,23,89]	Maximum Element: 89
Array: [5,11,3,20,8]	Maximum Element: 20

9. Divide and Conquer: Find Minimum Element

Write a program to find the minimum element from an array using divide and conquer.

Provide all required functions.

Sample Input	Sample Output
Array: [12,45,7,23,89]	Minimum Element: 7
Array: [8,2,10,4,6]	Minimum Element: 2

10. Divide and Conquer: Find Maximum and Minimum Together

Write a program to find both maximum and minimum elements of an array using a divide-and-conquer approach.

Provide all required functions.

Sample Input	Sample Output
Array: [4,8,1,9,3]	Maximum: 9 Minimum: 1
Array: [15,7,20,5,11]	Maximum: 20 Minimum: 5

11. Divide and Conquer: Second Largest Element

Write a program to find the second largest element in an array using a divide-and-conquer strategy.

Provide all required functions.

Sample Input	Sample Output
Array: [10,5,8,20,15]	Second Largest: 15
Array: [3,9,1,7,6]	Second Largest: 7

12. Divide and Conquer: Frequency Count

Write a program to count the frequency of a target element in a sorted array using a modified divide-and-conquer binary search approach.

Provide all required functions.

Sample Input	Sample Output
Array: [1,2,2,2,3,4], Target=2	Frequency: 3
Array: [1,1,2,3,3,3,4], Target=3	Frequency: 3

13. Divide and Conquer: Merge Two Sorted Arrays

Write a program to merge two sorted arrays into one sorted array using a divide-and-conquer strategy.

Provide all required functions.

Sample Input	Sample Output
Array1: [1,3,5], Array2:[2,4,6]	Merged Array: [1,2,3,4,5,6]
Array1:[2,7,9], Array2:[1,4,8]	Merged Array: [1,2,4,7,8,9]

14. Divide and Conquer: Maximum Difference Between Elements

Write a program to find the maximum difference between two elements where the larger element appears after the smaller element.

Provide all required functions.

Sample Input	Sample Output
Array: [2,3,10,6,4,8,1]	Maximum Difference: 8
Array: [7,9,5,6,3,2]	Maximum Difference: 2

15. Divide and Conquer: Merge K Sorted Arrays

Write a program to merge K sorted arrays into a single sorted array using divide-and-conquer.

Provide all required functions.

Sample Input	Sample Output
Arrays: [1,4,7], [2,5,8], [3,6,9]	Merged Array: [1,2,3,4,5,6,7,8,9]
Arrays: [1,5], [2,6], [3,4]	Merged Array: [1,2,3,4,5,6]

Part 2: Dynamic Programming Problems (15 Problems)

Instructions:

Write programs to solve the following problems using a dynamic programming approach.

The solution should identify overlapping subproblems and use memoization or bottom-up table construction where appropriate.

1. Dynamic Programming: Fibonacci Using Memoization

Write a program to find the n th Fibonacci number using a top-down dynamic programming approach. The program should store already calculated values to avoid repeated calculations.

Provide all required functions.

Sample Input	Sample Output
$n = 10$	Fibonacci Number: 55
$n = 15$	Fibonacci Number: 610

2. Dynamic Programming: Fibonacci Using Tabulation

Write a program to calculate the n th Fibonacci number using a bottom-up dynamic programming approach. Construct the solution using an iterative table.

Provide all required functions.

Sample Input	Sample Output
$n = 8$	Fibonacci Number: 21
$n = 12$	Fibonacci Number: 144

3. Dynamic Programming: Counting Arrays of 1s and 2s with Sum X

Given a positive integer X , count the number of different arrays consisting only of elements 1 and 2 such that the sum of all elements is exactly X . Use dynamic programming.

Provide all required functions.

Sample Input	Sample Output
--------------	---------------

X = 4	Number of Arrays: 5
X = 5	Number of Arrays: 8

4. Dynamic Programming: Climbing Stairs

A person wants to climb a staircase containing N steps. The person can climb either 1 step or 2 steps at a time. Find the total number of possible ways to reach the top using dynamic programming.

Provide all required functions.

Sample Input	Sample Output
N = 5	Number of Ways: 8
N = 7	Number of Ways: 21

5. Dynamic Programming: Coin Change Minimum Coins

Given different coin denominations and a target amount, find the minimum number of coins required to make the amount using dynamic programming.

Provide all required functions.

Sample Input	Sample Output
Coins: [1,3,4], Amount: 6	Minimum Coins: 2
Coins: [2,5,10,1], Amount: 27	Minimum Coins: 4

6. Dynamic Programming: Coin Change Number of Ways

Given different coin denominations and an amount, find the number of possible ways to make the given amount.

Provide all required functions.

Sample Input	Sample Output
Coins: [1,2,5], Amount: 5	Number of Ways: 4

Coins: [2,3,5], Amount: 7	Number of Ways: 2
---------------------------	-------------------

7. Dynamic Programming: 0/1 Knapsack Problem

Given weights and values of items, select items to maximize total value without exceeding the given capacity. Each item can either be selected or not selected.

Provide all required functions.

Sample Input	Sample Output
Weights: [2,3,4,5] Values: [3,4,5,6] Capacity: 5	Maximum Value: 7
Weights: [1,3,4] Values: [15,20,30] Capacity: 4	Maximum Value: 35

8. Dynamic Programming: 0/1 Knapsack with Item Selection

Modify the knapsack problem to display which items are selected along with the maximum achievable value.

Provide all required functions.

Sample Input	Sample Output
Weights:[2,3,4] Values:[4,5,7] Capacity:5	Selected Items: 1,2 Maximum Value: 9
Weights:[1,2,5] Values:[10,15,50] Capacity:3	Selected Items: 1,2 Maximum Value: 25

9. Dynamic Programming: Knapsack with Different Capacity

Solve 0/1 knapsack for a different capacity value.

Provide all required functions.

Sample Input	Sample Output
Weight:[2,4,5], Value:[10,20,30], Capacity:6	Maximum Value:40
Weight:[1,3,4], Value:[5,15,20], Capacity:5	Maximum Value:25

10. Dynamic Programming: Count Knapsack Solutions

Count the number of possible selections of items within a given capacity.

Provide all required functions.

Sample Input	Sample Output
Coins:[1,2,5], Quantity:[2,2,1], Amount:7	Possible:Yes
Coins:[2,4], Quantity:[1,1], Amount:10	Possible:No

11. Dynamic Programming: Limited Coin Change

Given limited coins, determine whether a target amount can be achieved.

Provide all required functions.

Sample Input	Sample Output
Coins:[1,2,5], Quantity:[2,2,1], Amount:7	Possible:Yes
Coins:[2,4], Quantity:[1,1], Amount:10	Possible:No

12. Dynamic Programming: Maximum Value Coin Selection

Select coins to maximize value under constraints.

Provide all required functions

Sample Input	Sample Output
Coins:[5,10,15], Limit:2	Maximum Value:25
Coins:[10,20,30], Limit:2	Maximum Value:50

13. Dynamic Programming: Fibonacci Pattern Matching

Find the nth value of a sequence where each value depends on previous two values.

Provide all required functions.

Sample Input	Sample Output
n=6	Result:8
n=9	Result:34

14. Dynamic Programming: Coin Change with Exact Coins

Find whether an amount can be formed using exactly K coins.

Provide all required functions.

Sample Input	Sample Output
Coins:[1,2,5], Amount:6, K=2	Possible:Yes
Coins:[2,4], Amount:7, K=3	Possible:No

15. Dynamic Programming: Number of Ways to Reach Nth Step

A person can climb 1, 2, or 3 steps at a time. Find the number of possible ways to reach the nth step using dynamic programming.

Provide all required functions.

Sample Input	Sample Output
N = 5	Number of Ways: 13
N = 6	Number of Ways: 24

Part 3: Greedy Algorithm Problems (15 Problems)

Instructions:

Write programs to solve the following problems using a greedy algorithm approach. The solution should make the locally optimal choice at each step to obtain the final optimized result.

1. Greedy Algorithm: Standard Activity Selection

You are given a set of activities where each activity has a start time and finish time. Select the maximum number of non-overlapping activities that can be performed by a person.

Provide all required functions.

Sample Input	Sample Output
Activities: (1,3), (2,5), (4,7), (6,9), (8,10)	Maximum Activities: 3
Activities: (0,6), (3,5), (5,7), (8,9)	Maximum Activities: 3

2. Greedy Algorithm: Modified Activity Selection with Priority

You are given activities with start time, finish time, and priority level. Select maximum activities while considering priority when two activities have the same finishing time.

Provide all required functions.

Sample Input	Sample Output
(1,4,A), (3,5,B), (5,7,A), (6,9,B)	Selected Activities: 3
(0,3,B), (2,5,A), (5,8,A)	Selected Activities: 2

3. Greedy Algorithm: Activity Selection with Priority

Each activity contains a start time, finish time, and priority value. Select maximum activities. If two activities have the same finishing time, choose the higher priority activity.

Sample Input	Sample Output
--------------	---------------

(1,4,5), (3,5,8), (5,7,3), (6,9,7)	Selected Activities: 3
(0,3,2), (2,5,9), (5,8,4)	Selected Activities: 2

4. Greedy Algorithm: Meeting Room Scheduling

You are given multiple meetings with their starting and ending times. Find the maximum number of meetings that can be arranged in one room without overlap.

Provide all required functions.

Sample Input	Sample Output
Meetings: (9,10), (9,12), (10,11), (11,13)	Maximum Meetings: 3
Meetings: (1,4), (2,3), (3,5), (6,8)	Maximum Meetings: 3

5. Greedy Algorithm: Interval Scheduling

Given a list of intervals, select the maximum number of non-overlapping intervals.

Provide all required functions.

Sample Input	Sample Output
Intervals: (1,3), (2,4), (3,5), (5,7)	Selected Intervals: 3
Intervals: (1,5), (2,3), (4,6), (7,9)	Selected Intervals: 3

6. Greedy Algorithm: Activity Selection with Maximum Duration Constraint

You are given activities and a maximum available time. Select activities using greedy selection without exceeding the time limit.

Sample Input	Sample Output
Activities: (1,3),(4,5),(6,8), Limit=7	Selected Activities: 2

Activities: (2,4),(5,6),(7,9), Limit=6	Selected Activities: 2
--	------------------------

7. Greedy Algorithm: Fractional Knapsack

You are given items with weights and values. Find the maximum profit that can be obtained when fractions of items are allowed.

Provide all required functions.

Sample Input	Sample Output
Weights: [10,20,30] Values: [60,100,120] Capacity: 50	Maximum Value: 240
Weights: [5,10,20] Values: [30,40,100] Capacity: 25	Maximum Value: 140

8. Greedy Algorithm: Fractional Knapsack with Item Selection

Modify fractional knapsack to display which items are selected and how much fraction of each item is taken.

Sample Input	Sample Output
Weight:[10,20,30], Value:[60,100,120], Capacity:50	Maximum Value:240
Weight:[5,15,25], Value:[50,90,100], Capacity:30	Maximum Value:180

9. Greedy Algorithm: Fractional Resource Allocation

A company has limited resources. Each project requires resources and provides profit. Allocate resources to maximize total profit. Partial allocation is allowed.

Provide all required functions.

Sample Input	Sample Output
Resources: [10,20,30] Profit: [50,90,120] Available: 40	Maximum Profit: 180
Resources: [5,15,25] Profit: [20,80,100] Available: 30	Maximum Profit: 160

10. Greedy Algorithm: Activity Selection with Attendance Flexibility

You are given a list of classes. Each class has a start time, end time, and class type.

- L (Lecturer): A student can arrive up to 5 time units late and still attend the class.
- P (Professor): A student must arrive exactly at the starting time.

A student can attend only one class at a time. Determine the maximum number of classes the student can attend using a greedy algorithm.

Provide all required functions.

Sample Input	Sample Output
n=3 (10,11,L) (10,11,P) (11,12,L)	Maximum Classes: 2
n=4 (1,4,L) (3,5,P) (0,6,L) (5,7,L)	Maximum Classes: 3

11. Greedy Algorithm: Activity Selection with Multiple Rooms

You are given a list of activities with start time and finish time. Unlike standard activity selection, there are multiple rooms available. Assign activities to rooms while maximizing the total number of selected activities.

Use a greedy approach based on activity finishing time.

Provide all required functions

Sample Input	Sample Output
Rooms = 2 Activities: (1,4),(2,3),(3,5),(5,7)	Maximum Activities: 4
Rooms = 1 Activities: (1,3),(2,5),(4,6),(7,9)	Maximum Activities: 3

12. Greedy Algorithm: Select Maximum Value Items

Given items where complete items can be selected, select items to maximize total value within capacity.

Sample Input	Sample Output
Values:[20,50,30], Capacity:2 Items	Maximum Value:80
Values:[40,10,60], Capacity:2 Items	Maximum Value:100

13. Greedy Algorithm: Maximum Events Selection

Given events with start and end days, find the maximum number of events that can be attended without conflict.

Provide all required functions.

Sample Input	Sample Output
Events:(1,2),(2,3),(3,4),(1,5)	Maximum Events: 3
Events:(1,4),(4,5),(5,6)	Maximum Events: 3

14. Greedy Algorithm: Budget Based Activity Selection

You have activities with cost and benefit. Select activities to maximize benefit within a budget.

Sample Input	Sample Output
Cost:[10,20,30], Benefit:[50,90,120], Budget:40	Maximum Benefit:180
Cost:[5,15,25], Benefit:[20,80,100], Budget:30	Maximum Benefit:160

15. Greedy Algorithm: Minimum Number of Activities

Given activities and a required time period, find the minimum number of activities needed to cover the maximum duration.

Sample Input	Sample Output
Activities:(1,4),(4,6),(6,9)	Minimum Activities:3
Activities:(0,5),(5,10)	Minimum Activities:2

16. Greedy Algorithm: Fraction Selection Problem

Select fractions of available resources to maximize profit using value/resource ratio.

Sample Input	Sample Output
Resources:[20,30,50], Profit:[100,120,200], Limit:60	Maximum Profit:280
Resources:[10,20,40], Profit:[50,80,160], Limit:50	Maximum Profit:210

17. Greedy Algorithm: Modified Fractional Knapsack

Items have weight, value, and priority. Select items based on combined value/weight ratio.

Sample Input	Sample Output
Weight:[10,20,30], Value:[60,100,120], Priority:[1,2,3], Capacity:50	Maximum Value:240
Weight:[5,15,25], Value:[30,80,100], Priority:[2,1,3], Capacity:30	Maximum Value:180